

Interpretación

```
public class Variable extends Simbolo {  
    private float valor = 0;  
  
    public Variable(String nombre, Tipo tipo) {  
        super(nombre, tipo);  
    }  
  
    public void setValor(float valor) {  
        this.valor = valor;  
    }  
  
    public float getValor() {  
        return valor;  
    }  
}
```

Para ejecutar el programa fuente, se requiere ejecutar cada una de las tuplas que representan los enunciados.

Lo primero que haremos es modificar la clase **Variable** que se utiliza para almacenar el nombre y tipo de las variables utilizadas en el programa, agregando un campo para almacenar el valor de la variable según se van ejecutando los enunciados.

```

public abstract class Tupla {
    protected int saltoVerdadero, saltoFalso;

    public Tupla(int sv, int sf) {
        saltoVerdadero = sv;
        saltoFalso = sf;
    }

    public void setSaltoVerdadero(int sv) {
        saltoVerdadero = sv;
    }

    public int getSaltoVerdadero() {
        return saltoVerdadero;
    }

    public void setSaltoFalso(int sf) {
        saltoFalso = sf;
    }

    public int getSaltoFalso() {
        return saltoFalso;
    }

    public String toString() {
        return this.getClass().getName() + ", " + saltoVerdadero + ", " + saltoFalso;
    }

    public abstract int ejecutar(TablaSimbolos ts);
}

```

Posteriormente, se modifica la superclase **Tupla** y todas sus subclases para implementar un método ejecutar, con argumento **TablaSimbolos**, que lleve a cabo la tarea correspondiente a cada enunciado.

```
public class Leer extends Tupla {
```

```
    Token variable;
```

```
    public Leer(Token variable, int sv, int sf) {
```

```
        super(sv, sf);
```

```
        this.variable = variable;
```

```
    }
```

```
    public String toString() {
```

```
        return "(" + super.toString() + ", [ " + variable + " ] )";
```

```
    }
```

```
public int ejecutar(TablaSimbolos ts) {  
    String valor = "0.0";  
  
    System.out.print("Da un valor para " + variable.getNombre() + " ");  
    BufferedReader entrada = new BufferedReader(new InputStreamReader(System.in));  
  
    try {  
        valor = entrada.readLine();  
    } catch (IOException ex) {}  
  
    Variable v = (Variable) ts.resolver(variable.getNombre());  
  
    try {  
        v.setValor(Float.parseFloat(valor));  
    } catch (NumberFormatException exception) {  
        System.out.println("Error: Número inválido");  
    }  
  
    return saltoVerdadero;  
}  
}
```

```
public class Escribir extends Tupla {
    Token cadena, variable;

    public Escribir(Token variableCadena, int sv, int sf) {
        super(sv, sf);

        if (variableCadena.getTipo().getNombre().equals(TipoToken.CADENA))
            cadena = variableCadena;
        else
            variable = variableCadena;
    }

    public Escribir(Token cadena, Token variable, int sv, int sf) {
        super(sv, sf);
        this.cadena = cadena;
        this.variable = variable;
    }

    public String toString() {
        if (variable == null)
            return "(" + super.toString() + ", [ " + cadena + " ] )";
        if (cadena == null)
            return "(" + super.toString() + ", [ " + variable + " ] )";

        return "(" + super.toString() + ", [ " + cadena + ", " + variable + " ] )";
    }
}
```

```
public int ejecutar(TablaSimbolos ts) {  
    if (cadena == null){  
        Variable v = (Variable) ts.resolver(variable.getNombre());  
        float valor = v.getValor();  
        System.out.println(valor);  
    } else if (variable == null) {  
        System.out.println(cadena.getNombre());  
    } else {  
        Variable v = (Variable) ts.resolver(variable.getNombre());  
        float valor = v.getValor();  
        System.out.println(cadena.getNombre() + valor);  
    }  
  
    return saltoVerdadero;  
}  
}
```

```

public class Asignacion extends Tupla {
    Token variable, valor1, valor2, operador;

    public Asignacion(Token variable, Token valor, int sv, int sf) {
        super(sv, sf);
        this.variable = variable;
        this.valor1 = valor;
    }

    public Asignacion(Token variable, Token valor1, Token operador, Token valor2,
        int sv, int sf) {
        super(sv, sf);
        this.variable = variable;
        this.valor1 = valor1;
        this.valor2 = valor2;
        this.operador = operador;
    }

    public String toString() {
        if (operador == null)
            return "(" + super.toString() + ", [ \"" + variable + ", \" +
                valor1 + "\" ] )";
        else
            return "(" + super.toString() + ", [ \"" + variable + ", \" +
                valor1 + ", \" + operador + ", \" + valor2 + \" ] ) ";
    }
}

```



```
public int ejecutar(TablaSimbolos ts) {  
    Variable v = (Variable) ts.resolver(variable.getNombre());  
    float operando1 = 0, operando2 = 0;  
  
    if (valor1.getTipo().getNombre().equals(TipoToken.NUMERO))  
        operando1 = Float.parseFloat(valor1.getNombre());  
    else  
        operando1 = ((Variable) ts.resolver(valor1.getNombre())).getValor();  
  
    if (valor2 != null)  
        if (valor2.getTipo().getNombre().equals(TipoToken.NUMERO))  
            operando2 = Float.parseFloat(valor2.getNombre());  
        else  
            operando2 = ((Variable) ts.resolver(valor2.getNombre())).getValor();  
}
```

```
if (operator == null)
    v.setValor(Float.parseFloat(valor1.getNombre()));
else
    switch (operator.getNombre()) {
        case "+": v.setValor(operando1 + operando2);
                  break;
        case "-": v.setValor(operando1 - operando2);
                  break;
        case "*": v.setValor(operando1 * operando2);
                  break;
        case "/": if (operando2 != 0)
                    v.setValor(operando1 / operando2);
                  else {
                      System.out.println("Error: División entre cero");
                      System.exit(1);
                  }
        break;
    }

return saltoVerdadero;
}
}
```

```
public class Comparacion extends Tupla {  
    Token valor1, valor2, operador;  
  
    public Comparacion(Token valor1, Token operador, Token valor2, int sv, int sf) {  
        super(sv, sf);  
        this.valor1 = valor1;  
        this.valor2 = valor2;  
        this.operador = operador;  
    }  
  
    public String toString() {  
        return "(" + super.toString() + ", [ " + valor1 + ", " + operador + ", " +  
            valor2 + " ] ) ";  
    }  
}
```

```
public int ejecutar(TablaSimbolos ts) {  
    float operando1 = 0, operando2 = 0;  
  
    if (valor1.getTipo().getNombre().equals("NUMERO"))  
        operando1 = Float.parseFloat(valor1.getNombre());  
    else  
        operando1 = ((Variable) ts.resolver(valor1.getNombre())).getValor();  
  
    if (valor2.getTipo().getNombre().equals("NUMERO"))  
        operando2 = Float.parseFloat(valor2.getNombre());  
    else  
        operando2 = ((Variable) ts.resolver(valor2.getNombre())).getValor();  
  
    switch (operator.getNombre()) {  
        case "<": return operando1 < operando2 ? saltoVerdadero : saltoFalso;  
        case "<=": return operando1 <= operando2 ? saltoVerdadero : saltoFalso;  
        case ">": return operando1 > operando2 ? saltoVerdadero : saltoFalso;  
        case ">=": return operando1 >= operando2 ? saltoVerdadero : saltoFalso;  
        case "==" : return operando1 == operando2 ? saltoVerdadero : saltoFalso;  
        case "!=": return operando1 != operando2 ? saltoVerdadero : saltoFalso;  
    }  
  
    return saltoVerdadero;  
}
```

```
}
```

```
public class FinPrograma extends Tupla {  
    public FinPrograma() {  
        super(-1, -1);  
    }  
  
    public String toString() {  
        return "(" + super.toString() + ", [ ], " + " )";  
    }  
  
    public int ejecutar(TablaSimbolos ts) {  
        return -1;  
    }  
}
```

```

public class PseudoInterprete {
    TablaSimbolos ts;

    public PseudoInterprete(TablaSimbolos ts) {
        this.ts = ts;
    }

    public void interpretar(ArrayList<Tupla> tuplas) {
        int indiceTupla = 0;
        Tupla t = tuplas.get(0);

        do {
            indiceTupla = t.ejecutar(ts);
            t = tuplas.get(indiceTupla);
        } while (!(t instanceof FinPrograma));
    }
}

```

Para ejecutar las tuplas se implementa la clase **PseudoInterprete**, la cual solo requiere el conjunto de tuplas y la tabla de símbolos.

Para ejecutar el intérprete con todas su fases (análisis léxico, análisis sintáctico, generación de representación intermedia e interpretación, incluyendo la tabla de símbolos) extendemos la clase de prueba que se ha utilizado anteriormente.

```
public class PruebaInterprete {  
    public static void main(String[] arg) throws LexicalException, SyntaxException {  
        String entrada = leerPrograma("/.../ejemplo.alg");  
        PseudoLexer lexer = new PseudoLexer();  
        lexer.analizar(entrada);  
  
        System.out.println("*** Análisis léxico ***\n");  
  
        for (Token t: lexer.getTokens())  
            System.out.println(t);  
  
        System.out.println("\n*** Análisis sintáctico ***\n");  
  
        TablaSimbolos ts = new TablaSimbolos();  
        PseudoGenerador generador = new PseudoGenerador(lexer.getTokens());  
  
        PseudoParser parser = new PseudoParser(ts, generador);  
        parser.analizar(lexer);  
    }  
}
```

Para ejecutar el intérprete con todas su fases (análisis léxico, análisis sintáctico, generación de representación intermedia e interpretación, incluyendo la tabla de símbolos) extendemos la clase de prueba que se ha utilizado anteriormente.

```
System.out.println("\n*** Tabla de símbolos ***\n");

for (Simbolo s: ts.getSimbolos())
    System.out.println(s);

System.out.println("\n*** Tuplas generadas ***\n");

for (Tupla t: generador.getTuplas()) {
    System.out.println(t);
}

System.out.println("\n*** Ejecucion del programa ***\n");

PseudoInterprete interprete = new PseudoInterprete(ts);
interprete.interpretar(generador.getTuplas());
}
```



```
inicio-programa
  leer numeroDeElementos
  promedio = 0
  i = 0

  mientras (i < numeroDeElementos)
    leer elemento
    promedio = promedio + elemento
    i = i + 1
  fin-mientras

  promedio = promedio / numeroDeElementos
  escribir "El promedio es ", promedio
fin-programa
```

*** Ejecucion del programa ***

```
Da un valor para numeroDeElementos 9
Da un valor para elemento 10
Da un valor para elemento 8
Da un valor para elemento 7
Da un valor para elemento 9
Da un valor para elemento 5
Da un valor para elemento 9
Da un valor para elemento 6
Da un valor para elemento 10
Da un valor para elemento 8
El promedio es 8.0
```